# WRONG NUMBER

## Complete Gameplay Loop & Systems Specification

**Genre:** Cooperative Multiplayer / AI-Powered Emergency Dispatch / Comedy / Resource Management
**Engine:** Bevy / Rust
**Players:** 1-6
**Perspective:** First-person, free-look camera
**Shift Duration:** 10 minutes
**Core Gameplay:** Answer AI-generated emergency calls, determine their legitimacy, allocate limited emergency resources, and survive the consequences of your decisions.

### User-supplied design brief / 27 October 2026

Reference specification, not a list of completed features. The source text is preserved alongside this PDF. The supplied text ends mid-sentence in section 11.2; no missing requirements have been invented.

Implementation precedence: explicit user changes take priority. Keep chairs unnumbered, use E to pop into seats, keep daylight outside and retain simple low-poly graphics.

# 1. GAMEPLAY OVERVIEW

Wrong Number is a cooperative multiplayer emergency dispatch simulator in which up to six players operate a dispatch center inside an older New York-style brick building.

Each player occupies a physical workstation containing a computer, telephone, and chair. The office overlooks a dense urban environment, with neighboring buildings visible through tall arched windows.

At the front of the room is a massive shared display showing a live 2D city map.

The map displays emergency service headquarters, hospitals, active incidents, emergency vehicle positions, and dispatch routes.

The game operates in repeating ten-minute shifts.

During each shift, players receive calls from AI-controlled citizens reporting emergencies, suspicious activities, complaints, and occasionally completely fabricated situations.

Players must speak naturally with callers through their microphones to determine whether the reported incident is genuine, exaggerated, insignificant, or an outright prank.

After investigating the call, players choose whether to dispatch emergency resources or decline the request.

Dispatching resources consumes temporarily limited emergency capacity.

Refusing a legitimate emergency can cause property damage, injuries, additional emergencies, and financial penalties.

Correctly declining a prank or non-emergency preserves resources and contributes positively to the team's performance.

The game tracks the consequences of every decision.

At the end of the ten-minute shift, players gather for a performance review featuring statistics, incident summaries, and recordings of the most ridiculous conversations.

Players receive a grade and funding that can be spent on upgrades for subsequent shifts.

The game should be mechanically challenging but socially entertaining, with unpredictable interactions and emergent comedy.

# 2. PHASE ONE - START OF DAY

## 2.1 Player Spawn

At the beginning of each shift, all players spawn inside the dispatch center's break room.

The break room is physically connected to the main dispatch office.

It contains a refrigerator, microwave, coffee machine, tables, chairs, cabinets, and miscellaneous interactive props.

All players begin standing and can move freely.

No player begins automatically seated at a dispatch workstation.

The room should feel like a casual employee gathering area before work begins.

## 2.2 Ten-Minute Shift Timer

A ten-minute countdown begins when the shift starts.

The timer represents the active emergency dispatch period.

**The timer must be synchronized across all players and controlled by the authoritative server.**

The countdown should be visible on an office clock, workstation UI, or another appropriate display.

The timer should not pause when players are walking around, talking, or refusing to sit down.

The shift continues whether players are prepared or not.

This creates opportunities for players to waste time, ignore incoming calls, or rush back to their workstations.

### 2.3 Moving to Workstations

Players can leave the break room and enter the main dispatch area.

There are six workstation chairs positioned behind a long shared desk.

Each workstation is individually numbered.

Players may approach any unoccupied chair.

When the player looks at a chair, the interaction reticle changes appearance to indicate that the object is interactable.

The player can click or press the interaction key to sit.

Workstation ownership must be synchronized so two players cannot occupy the same chair simultaneously.

Players are not required to sit immediately.

A player who remains in the break room may miss calls or force teammates to handle additional workload.

## 3. FIRST-PERSON CAMERA AND INTERACTION SYSTEM

### 3.1 Free-Look Camera

The game uses a first-person camera.

Players should be able to look freely around the environment using their mouse.

The camera must support natural vertical and horizontal rotation.

The camera should remain responsive whether the player is standing or sitting.

### 3.2 Persistent Circular Reticle

A small circular reticle should remain visible at the center of the screen throughout normal gameplay.

The reticle serves as the primary interaction indicator.

Default appearance:

- Small circle.
- Neutral color.
- Minimal visual obstruction.
- Consistent size.

When the player looks at an interactable object within interaction range, the reticle should change color, brightness, or shape.

For example:

- White: no interaction available.
- Green: valid interaction available.
- Yellow: interaction available but requires a condition.
- Red: unavailable or locked interaction.

The specific colors should be configurable.

The system should use a raycast from the player's camera to determine which object is being targeted.

## 3.3 Interactable Objects

Initial interactable objects include:

- Workstation chairs.
- Computer monitors.
- Telephones.
- Break-room doors.
- Meeting-room doors.
- Selected office props.
- Break-room equipment.

Each interactable object should provide a contextual interaction prompt.

For example:

"Press E to Sit"

"Press E to Answer Call"

"Press E to Open Door"

The interaction system should be reusable across the entire game.

## 3.4 Sitting Down

When a player interacts with an available workstation chair:

1. The server validates that the chair is unoccupied.
2. The chair becomes assigned to that player.
3. The player transitions into a seated position.
4. The camera moves smoothly to the seated eye position.
5. The player retains free-look camera movement.
6. The workstation becomes available for interaction.

The player must be able to look around while seated.

They should be able to:

- Look down at their computer.
- Look up at the giant wall map.
- Look left and right at other dispatchers.

- Look toward the windows.
- Observe environmental events.

The camera should not be permanently locked onto the monitor.

### 3.5 Computer Interaction

While seated, the player can look at their monitor and interact with it.

The computer should support an interaction or focus mode.

In focus mode, the player can use the dispatch interface.

When exiting focus mode, the player returns to normal seated free-look.

The player should also be able to stand up and leave the workstation.

Standing up releases the chair assignment unless the game explicitly supports temporary reservations.

## 4. THE SHARED CITY MAP

### 4.1 Giant Wall Display

A large display is mounted at the front of the dispatch office.

All six players can see it from their chairs.

The display shows a dark-mode 2D map of the fictional city.

The map is based on the existing road network, which contains intersections, connected roads, named streets, and dispatchable addresses.

The wall display should remain relatively simple.

It should not show detailed caller biographies or long descriptions.

Its purpose is to provide a shared overview of the city's emergency activity.

### 4.2 Initial Emergency Locations

For the first playable version, the city should have three primary emergency service locations:

**Police Headquarters (HQ)**

The starting location for police units.

**Fire Headquarters**

The starting location for fire engines.

**Hospital / EMS Headquarters**

The starting location for ambulances.

Each location must correspond to a real node or accessible location in the road graph.

These locations should appear as permanent map markers.

### 4.3 Incident Markers

When a call is received, the corresponding incident location appears on the map.

The marker should indicate the reported incident location, not necessarily its true location.

If the caller provides an incorrect address, the marker may initially be inaccurate.

Markers should use simple colored icons or dots.

Examples:

- Red: urgent incident.
- Orange: lower-priority incident.
- Blue: police.
- Yellow: fire.
- Green: ambulance.

The shared map should also show emergency vehicles moving toward incidents.

### 4.4 Road-Based Movement

Emergency vehicles must follow the actual road graph.

Do not move vehicles in straight lines between headquarters and incidents.

Use Dijkstra's algorithm or A* to calculate routes.

The route should account for:

- Road distance.
- Road speed.
- Current road availability.
- Unit movement speed.
- Road closures where implemented.

The calculated route should be displayed on the map.

Emergency vehicle markers should move along that route over time.

All clients must observe the same authoritative vehicle positions.

## 5. INCOMING CALL SYSTEM

### 5.1 Initial Call Delay

After a player sits at a workstation, their first incoming call should arrive after a randomized delay of **1-10 seconds**.

This delay should be controlled by the server.

The delay begins when the player becomes eligible to receive calls.

Each occupied workstation may receive calls independently.

The system should prevent a player from receiving multiple simultaneous active conversations unless a future upgrade explicitly enables call waiting or multiple lines.

### 5.2 Incoming Call Notification

When a call arrives:

1. The workstation telephone rings.

2. The computer displays an incoming-call notification.
3. The player's workstation is marked as ringing.
4. The player may answer the call.
5. The AI caller session begins.

The telephone should produce spatial audio so nearby players can hear it ringing.

The notification should be visually noticeable without completely obscuring the screen.

### 5.3 Answering the Call

When the player answers:

- The ringing stops.
- The caller begins speaking.
- The call timer starts.
- The caller's reported information appears on the workstation.
- The player can communicate through their microphone.

The AI caller should behave like a person reporting an emergency or complaint.

The caller may be frightened, confused, angry, dishonest, sarcastic, or completely ridiculous.

The player should be able to ask questions naturally.

The AI should respond based on its character biography, personality, incident scenario, and conversation history.

## 6. AI CALLER CONVERSATION SYSTEM

### 6.1 Primary Gameplay Objective

The central gameplay challenge is determining whether the caller's request justifies emergency dispatch.

Players must use conversation to assess the situation.

The game should include:

- Genuine emergencies.
- Prank calls.
- Exaggerated emergencies.
- Minor complaints.
- Misunderstood situations.
- Serious emergencies disguised as minor problems.
- Ambiguous incidents.

Not every call should have an obvious correct answer.

### 6.2 Hidden Incident Truth

Every generated call must have an underlying authoritative scenario.

The scenario determines:

- Whether the emergency is genuine.

- The actual severity.
- The true location.
- The incident's potential consequences.
- The caller's knowledge.
- Whether the caller is lying.
- Whether the situation can escalate.
- What resources are appropriate.

This information must be established before the conversation begins.

The LLM should not invent a different underlying reality midway through the call.

The AI controls the caller's dialogue and permitted behavior.

The simulation controls the actual incident outcome.

### 6.3 Voice Conversation Pipeline

The system should use:

Player microphone → Speech-to-text → LLM conversation → Text-to-speech → Caller audio.

Prefer local speech recognition and local text-to-speech where practical.

Use an inexpensive cloud LLM for conversational intelligence.

The game should support configurable AI providers and models.

The caller's dialogue must remain consistent with the character's personality.

### 6.4 Call Information

The workstation should display:

- Caller name, if known.
- Caller phone number or identifier.
- Reported address.
- Incident type, if identified.
- Call duration.
- Conversation notes.
- Current call status.
- Available emergency resources.

The player should not automatically receive the hidden truth.

The player must investigate through conversation.

## 7. DISPATCH DECISION SYSTEM

### 7.1 Available Decisions

During or after a conversation, the player can choose:

**Send Police**

Dispatch an available police unit.

**Send Fire**

Dispatch an available fire engine.

**Send Ambulance**

Dispatch an available ambulance.

**Send Full Response**

Dispatch one available unit from each of the three emergency service categories.

**Decline Dispatch**

Refuse to send emergency resources.

The UI should allow players to inspect current resource availability before committing.

## 7.2 Decision Confirmation

When a player selects a dispatch option:

1. Validate resource availability.
2. Confirm the requested destination.
3. Reserve the selected units.
4. Create dispatch assignments.
5. Calculate routes.
6. Update the shared city map.
7. Update resource availability.
8. End the call.

For the initial version, selecting a final decision should automatically close the call.

Later versions may allow players to keep callers on the line while emergency units respond.

## 7.3 Declining Dispatch

When the player selects "Decline Dispatch":

1. The call ends.
2. No emergency units are consumed.
3. The underlying incident continues or resolves according to its scenario.
4. Potential consequences are calculated.
5. The decision is recorded for the performance review.

Declining a prank or harmless complaint may produce no damage.

Declining a legitimate emergency may result in significant damage.

However, consequences should depend on the scenario and not be automatically catastrophic.

# 8. DAMAGE AND CONSEQUENCE SYSTEM

## 8.1 Damage Calculation

Every incident should support a damage or consequence model.

Potential consequences include:

- Property damage.
- Injuries.
- Infrastructure damage.
- Additional emergencies.
- Public safety penalties.
- Financial losses.
- Reduced department reputation.

For the first version, use an abstract monetary damage value.

Possible outcomes range from zero dollars to millions of dollars.

Examples:

A fabricated emergency may result in $0 damage.

A minor incident may produce $500 in damage.

An unattended structure fire may cause hundreds of thousands of dollars in damage.

A catastrophic city incident may cause millions.

These values are gameplay balancing parameters, not real-world damage estimates.

## 8.2 Incident Outcomes

The simulation should calculate consequences using:

- Incident type.
- Severity.
- Response decision.
- Units assigned.
- Response time.
- Escalation deadline.
- Relevant city conditions.
- Randomized scenario factors.

Use deterministic random seeds where necessary to keep multiplayer clients synchronized.

## 8.3 Damage Timing

Damage should not always be applied immediately when a player declines.

Some incidents may resolve harmlessly.

Others may escalate after a delay.

For example, declining a genuine fire may lead to increasing damage over the following minute.

This allows consequences to overlap with later calls.

### 8.4 Event Feedback

Major incidents should generate observable effects.

An explosion may produce:

- A bright flash outside.
- A distant explosion sound.
- Building camera shake.
- Flickering lights.
- A new map incident marker.
- Additional incoming calls.

A shooting incident may produce:

- Distant gunfire.
- Police radio chatter.
- Emergency sirens.
- New incident markers.

A street-racing incident may produce:

- Engine sounds.
- Tire squeals.
- Police sirens.
- Moving incident markers.

The game should calculate the perceived intensity of sounds and environmental effects based on the incident's distance from the dispatch center.

## 9. EMERGENCY RESOURCE SYSTEM

### 9.1 Shared Resources

All players share the same emergency resource pool.

Initial resource categories:

- Police.
- Fire.
- Ambulance.

Each category has a limited number of available units.

The resource counts should be visible on workstation screens.

The game should also display resource availability on the shared map when appropriate.

### 9.2 Dispatching Units

When a player sends a unit:

1. The available resource count decreases.
2. The unit leaves its headquarters.
3. The map shows the unit traveling.
4. The unit arrives at the incident.
5. The incident begins resolving.
6. The unit returns to headquarters.
7. The resource count increases when the unit becomes available again.

### 9.3 Return Time

For the initial prototype, the target recovery period after incident handling should be approximately **10-20 seconds**.

This should be configurable.

However, the total unavailability time must include travel to the incident, on-scene handling, and return travel.

Do not instantly return a unit to headquarters after the incident is resolved.

The unit should visibly travel back along the road network.

### 9.4 Resource Display

Each workstation should show something similar to:

Police: 3 / 5 available
Fire: 1 / 3 available
Ambulance: 2 / 4 available

When units are dispatched, these values should update immediately.

The system must prevent multiple players from simultaneously dispatching the same unit.

The server should handle resource reservations atomically.

### 9.5 Full Response

Selecting "Send Full Response" attempts to dispatch police, fire, and ambulance units.

The server must verify that the necessary units are available.

If resources are insufficient, the UI should display which units are unavailable.

Do not silently create additional units.

The player may choose a partial response or cancel the action.

## 10. REPEATING CALL LOOP

### 10.1 Next Call Delay

After a player completes a call by dispatching or declining:

1. The active call ends.

2. The workstation returns to an idle state.
3. A randomized delay begins.
4. Another call arrives after **1-15 seconds**, provided the shift remains active.

This cycle continues until the shift timer reaches zero.

The delay should be configurable and adjusted for difficulty.

### 10.2 Simultaneous Calls

Multiple players may receive calls simultaneously.

Each player's conversation is independent, but all calls affect the same city and emergency resource pool.

For example:

Player 1 may dispatch the final available ambulance.

Player 2 may then receive a legitimate medical emergency.

Player 2 must decide how to handle the situation without an available ambulance.

This creates the resource-management challenge.

### 10.3 Workload Balancing

The call director should control the overall volume and severity of incoming calls.

It should consider:

- Number of active players.
- Current shift difficulty.
- Available resources.
- Active incidents.
- Current city conditions.
- Remaining shift time.

Avoid overwhelming a solo player with the same workload intended for six players.

However, the game should not guarantee that sufficient resources are always available.

Scarcity and difficult decisions are intentional.

## 11. LIVE CITY AUDIO AND VISUAL FEEDBACK

### 11.1 Ambient City Sounds

The dispatch office should have an ambient urban soundscape.

Possible sounds include:

- Distant traffic.
- Car horns.
- Sirens.
- Helicopters.

- Pedestrians.
- Construction.
- Rain.
- Wind.
- Occasional street activity.

These sounds should be spatialized relative to the office windows and exterior environment.

## 11.2 Incident-Driven Audio

Important city events should produce audio feedback.

Examples:

**Explosion**

A sudden distant boom, window rattle, and room shake.

**Gunfire**

Distant gunshots and subsequent emergency activity.

**Vehicle pursuit**

High-speed engine sounds, tire squeals, and police sirens.

**Structure fire**

Sirens, distant crackling, and radio communications.

**Emergency vehicle dispatch**

Sirens become audible as nearby units